A deterministic emulator of the Veros ACC last-year state

28 October 2026

Summary

- Goal: predict the last-year temperature of the Veros ACC channel directly from the EKE-closure coefficients (c_k, c_ϵ), skipping the 100-year integration.
- Method: a small deterministic network $z \mapsto 20$ leading PCA coefficients of the training states, $z = (\log c_k, \log c_\epsilon)$ standardised, averaged over 5 initialisations. It fits on the run states in seconds on a CPU, with no GPU and no sampling.
- Interpolation (scattered points, a band of three rows, the 5×5 and 7×7 centre blocks, leave-one-out): RMSE 0.016–0.037 K. The nearest training run gives 0.038–0.118 K and the best classical emulator, a Gaussian process on one collapsed parameter s, 0.019–0.046 K.
- Extrapolation (top row, outer 51, outer 75): 0.059 / 0.068 / 0.123 K, against 0.079 / 0.075 / 0.125 K for the GP on s and 0.176 / 0.134 / 0.159 K for the nearest run. The warm corner sets the error: 0.15 K one row from the training set, 1.06 K three cells away.
- With only four training runs it loses to the GP on s (0.139 vs 0.120 K).

1 Data

- 100 Veros ACC runs on a 10×10 log-spaced grid, $c_k \in [0.0125, 0.8]$, $c_\epsilon \in [0.0875, 5.6]$ (ratio $2^{2/3}$); 100 years, 30-day output; interior grid $30 \times 42 \times 15$ (x, y, z) with a 930-cell ridge.
- Target: $y_r \in \mathbb{R}^{17970}$, the mean temperature of the last 12 outputs (360 days) of run r on the water cells. $S = 35$ psu everywhere, so it is dropped.
- The runs still drift, so the averaging window matters. Over the last 20 years, the RMS change from the first to the last year is 0.16 K, and the last-year mean differs from the 20-year mean by 0.08 K.
- The two parameters act through one combination, $s = \log c_k - 1.08 \log c_\epsilon$ (Figure 1). Below $s \approx -2$ (48 runs) the domain-mean T stays within 0.09 K; above, it rises by 1.1 K.

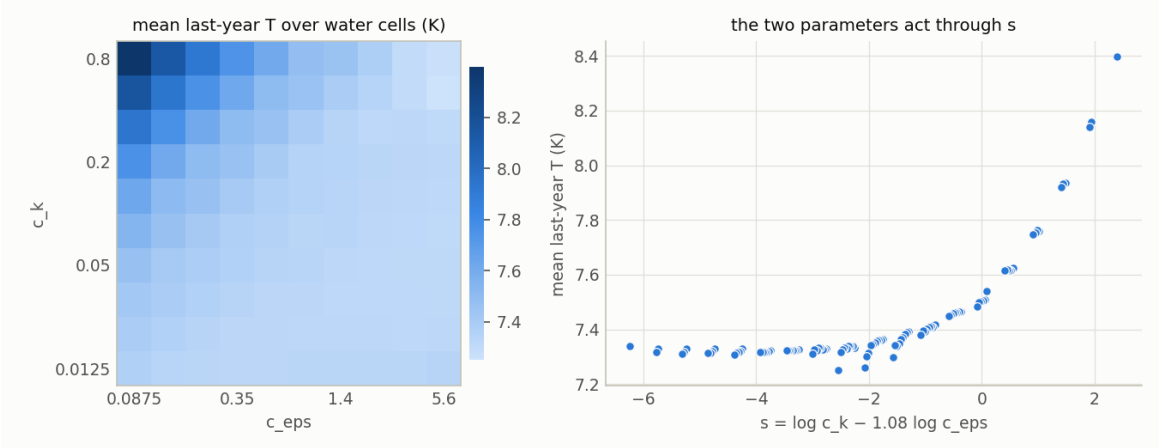


Figure 1: Mean last-year T over the water cells: over the parameter grid (left) and against s (right), the direction along which the first PCA coefficient of the states varies linearly.

2 Method

- Input z: $(\log c_k, \log c_\epsilon)$ standardised over the design.
- Output: a field in the span of the training states,

$$\hat{y}(z) = \mu + \sum_{k=1}^{K} a_k(z)\, v_k, \qquad K = 20, \tag{1}$$

  where μ and v_k are the mean and the leading principal directions of the training states.
- Network $a(z)$: a multilayer perceptron $2 \to 64 \to 64 \to K$ (scikit-learn, L-BFGS), with targets $a_k(z_r) = v_k^\top (y_r - \mu)$ divided by $\mathrm{std}(a_1)$. With orthonormal v_k, the loss is then the physical MSE of the projected field up to a constant:

$$\mathcal{L} = \frac{1}{n} \sum_{r=1}^{n} \frac{\|a(z_r) - V^\top (y_r - \mu)\|^2}{\mathrm{std}(a_1)^2} + \frac{\alpha}{n} \|W\|^2. \tag{2}$$

- The prediction is the mean of 5 networks that differ only in their initialisation.
- (activation, α) $\in \{\mathrm{ReLU}, \tanh\} \times \{10^{-4}, \ldots, 1\}$ is chosen by 5-fold cross-validation on the training runs of each split. The choice is ReLU with $\alpha = 10^{-4}$ to 10^{-2} everywhere except two splits: tanh on the 7×7 block, and $\alpha = 1$ with four training runs, where the cross-validation is not meaningful. The scores are flat for $\alpha \leq 10^{-2}$.
- Fitting on run means rather than on snapshots changes nothing: $\sum_t \|f(z_r) - x_{r,t}\|^2 = n_t \|f(z_r) - \bar{x}_r\|^2 + \mathrm{const}$.
- Baselines:
  - the training mean;
  - the nearest training run in z;
  - thin-plate RBF interpolation of every cell;
  - a GP per PCA coefficient on z (Matérn 5/2, one length scale per input, white noise);
  - the same GP on $s = w^\top z$, with w from a linear fit of a_1 on z.

3 Evaluation

- Eight hold-out splits of the grid, plus leave-one-out over the 100 runs:
  - interpolation: 10 scattered interior runs; the rows $c_k = 0.126$, 0.2, 0.3175; the centre 5×5 and 7×7 blocks;
  - extrapolation: the top row ($c_k = 0.8$); the 51 or 75 outer runs, training on the centre;
  - four training runs, one step in from each corner.
- Metric: RMSE over the water cells against the last-year mean, per held-out run, then the mean and median over the runs.
- Cost: a fit takes seconds on one core and a prediction microseconds. The whole evaluation takes 25 min on 8 cores, mostly the hyper-parameter search.

split	held out / train	network (median)	GP on s	GP on z	RBF	nearest run	mean state
scattered	10 / 90	**0.016** (0.013)	0.019	0.022	0.021	0.038	0.161
band (three rows)	30 / 70	**0.037** (0.032)	0.046	0.051	0.057	0.118	0.204
centre 5×5	25 / 75	**0.026** (0.023)	0.029	0.037	0.037	0.080	0.162
centre 7×7	49 / 51	**0.035** (0.032)	0.036	0.070	0.068	0.116	0.203
leave-one-out	100 / 99	**0.024** (0.017)	0.028	0.032	0.034	0.085	0.221
top row	10 / 90	**0.059** (0.033)	0.079	0.083	0.084	0.176	0.482
outer 51	51 / 49	**0.068** (0.028)	0.075	0.088	0.093	0.134	0.265
outer 75	75 / 25	**0.123** (0.057)	0.125	0.164	0.138	0.159	0.226
four runs	96 / 4	0.139 (0.127)	**0.120**	0.234	0.146	0.181	0.260

Table 1: RMSE (K) of the last-year temperature over the water cells, mean over the held-out runs; for the network also the median. Bold: lowest mean per split.

4 Findings

- **The network is best in 8 of 9 evaluations** (Table 1, Figure 2). In interpolation its error is half or less of the nearest run's.
- **Inside the grid it barely beats the GP on s.** On the 7×7 block it gets 0.035 vs 0.036 K and wins 26 runs of 49; in leave-one-out it wins 61 of 100. Since the parameters act almost only through s, a one-dimensional GP is a strong reference.
- **Its gain is at the grid edges.** On the top row it gets 0.059 vs 0.079 K, and at the corner run 0.15 vs 0.37 K. In leave-one-out, its largest errors are at the two high-c_k corners: 0.16 and 0.12 K (Figure 3).
- **The error sits in the upper** 500 m (Figure 4). In interpolation it is 0.02–0.05 K there and 0.01–0.02 K below 1000 m.
- **Large gaps defeat it.** On outer 75 (25 training runs) its median is 0.057 vs 0.038 K for the GP on s. With four runs it gets 0.139 vs 0.120 K, and in the flat regime the nearest run is better still (median 0.064 K).
- **The warm corner** ($c_k = 0.8$, $c_\epsilon = 0.0875$) is the hardest state. Three cells from the training set (outer 75), it is off by 1.06 K: a cold bias of -0.71 K, reaching -6.4 K locally (Figure 5). Every method is above 1 K there.
- **Part of that is the basis.** Projecting the held-out truth on the 20 training principal directions already leaves 0.010–0.017 K in interpolation and 0.031–0.079 K in extrapolation. On outer 75 that is 0.079 K of the network's 0.123 K.
- **The spread over initialisations is not an uncertainty.** It is 0.005–0.016 K in interpolation and 0.013–0.024 K in extrapolation, while the error grows from 0.016 to 0.139 K.

5 Next steps

- Inputs $(s, s_\perp)$ instead of $(\log c_k, \log c_\epsilon)$, to build in the collapse that makes the GP on s strong with few runs.
- For extrapolation, either a decoder not restricted to the span of the training states, or more runs near the warm corner.
- An uncertainty from resampling the training runs rather than the initialisations.
- Fixed hyper-parameters (ReLU, $\alpha = 10^{-3}$): the cross-validation scores are flat, and the evaluation would run about 50 times faster.

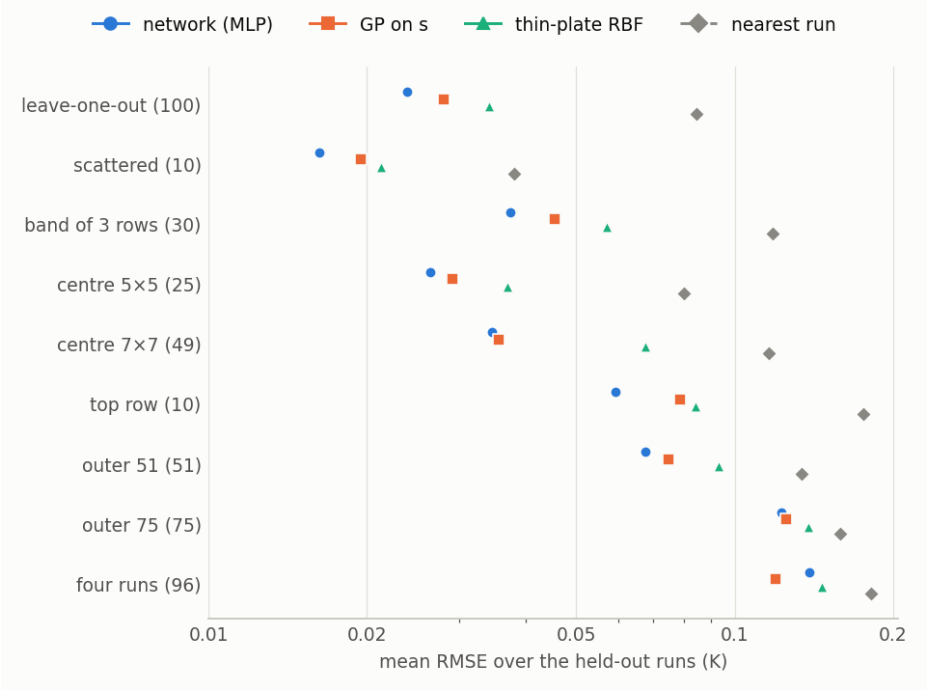


Figure 2: Mean RMSE over the held-out runs per split (log scale), for the network and the three best baselines; held-out run counts in brackets.

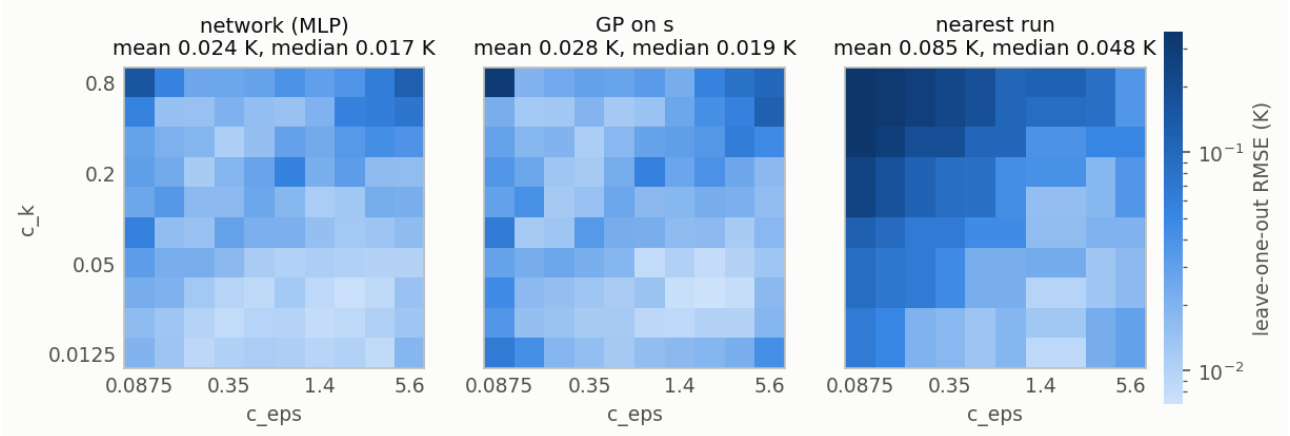


Figure 3: Leave-one-out RMSE of every run over the parameter grid (each run predicted from the 99 others).

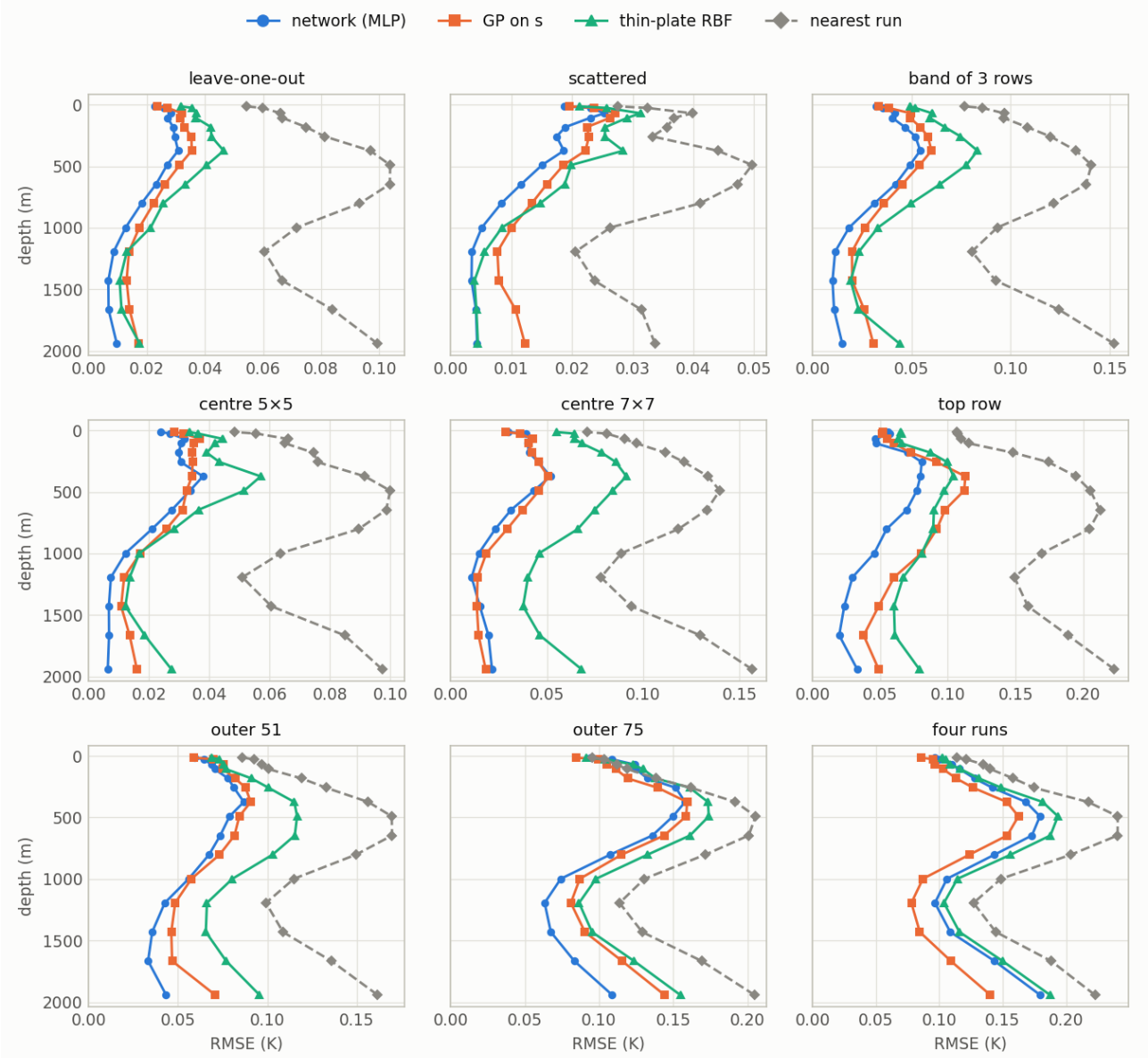


Figure 4: RMSE by depth, mean over the held-out runs of each split.

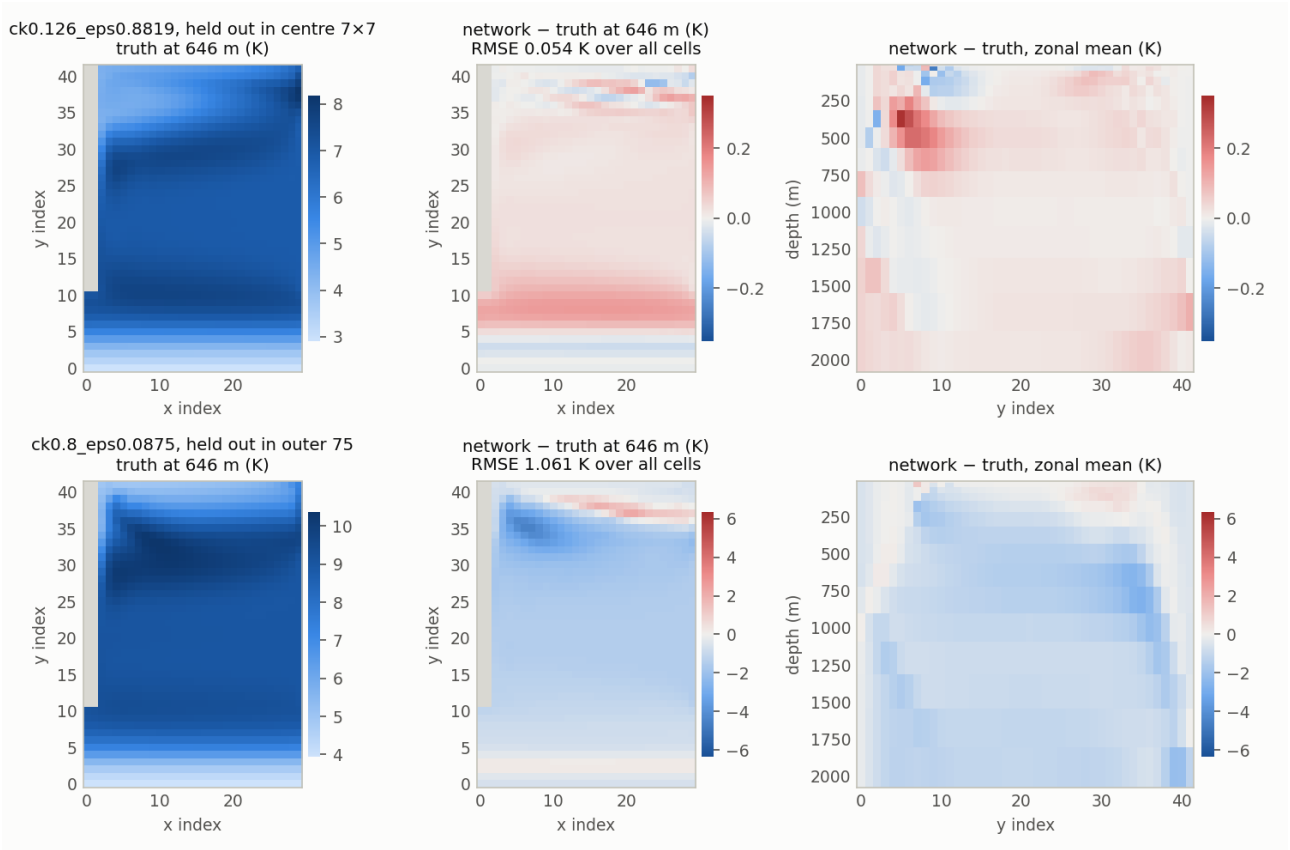


Figure 5: Truth at 646 m (left), network error at 646 m (centre) and zonal mean of the error (right). Top: the centre run of the 7×7 block, held out with the block (RMSE 0.054 K). Bottom: the warm corner run, held out with the outer 75 (RMSE 1.06 K).